STANDALONE EDITION / VERIFIED RETRAINING

Standalone book companion checklist and retraining audit report

Results reviewed on 2026-10-29 (Asia/Shanghai); English edition prepared on 2026-11-02. This report describes the new workspace and results of standalone v1.0.0. Version 1.0.1 translates the presentation and documentation; the original frozen project and v1.0.0 artifacts remain available.

All 156 formal neural training runs, 8 FEM runs, new chapter evaluations, and independent audits completed. The pipeline exited with code 0; final re-verification passed for 3,364 artifact identities, and all 9,550 original frozen files were unchanged. Completion refers to the agreed companion scope, not implementation of every manuscript algorithm or convergence of every method to one accuracy level.

1 Delivery scope and audit findings

Part	Formal scope	Delivery and limitations
Chapter 2	18 neural runs + 8 FEM runs	PINN, Deep Ritz, norm-aware VPINN; 1D/2D Poisson. Final-iterate models, independent quadrature, grid refinement, and reload checks.
Chapter 3	57 runs; 54 direct predictors + 3 residual models	Four operators, POD, physical FNO, VarMiON, HINTS; new fixed/variable-forcing tests; 448 hybrid solves.
Chapter 4 baseline	48 runs; 72 model stages	16 workflows x 3 seeds, all trained from scratch. New 64-instance family test; online-control comparisons use the same first 8 cases.
Chapter 4 extensions	12 runs	Nonlinear Deep Uzawa and three sparse-observation FEM-NN settings; 512 new observation locations and independent PDE diagnostics.
Chapter 5	21 runs; 21 selected models	7 workflows x 3 seeds; 128 new cases; control and state constraints checked separately.
Chapters 1, 6	Explanatory companion material	Retains the manuscript mapping and methodological limitations without inventing training tasks.

Checks passed: 5 packaging tests, 334 numerical regression tests, Python optimized-mode checks, 21 small-budget Chapter 3 training/reload checks, and formal budget, freeze, independent-evaluation, checkpoint-reload, and physical-metric checks for every chapter. All 317 source-manifest entries matched. The locally synchronized 2,426 evidence files and source copies passed SHA256 verification.

Budgets were not reduced: Chapter 2 uses 5000 Adam updates and a nominal 1000 L-BFGS iterations per run. Chapter 3 baseline/POD/VarMiON v1 use 1600 epochs; physical FNO and HINTS use 400; VarMiON v2 uses 800. Chapters 4-5 retain their formal configurations; Chapter 5 family operators use 300 epochs / 4800 updates, and ADMM uses 30 outer iterations. Existing stopping rules can terminate L-BFGS below its nominal cap; actual counts are recorded.

Fixed rules: Chapter 2 retains the final iterate; Chapters 3-5 select using train/val only. HINTS initializes from the same-seed VarMiON newly trained in this workspace. Old formal weights were not used. These tests have been viewed: subsequent development must use train/val, and reuse of these tests is a retest.

2 Results and interpretation

2.1 Chapter 2: solution errors and FEM references

Relative errors below are dimensionless values, not percentages. Each row has 3 seeds; H1 denotes the gradient seminorm.

Dim.	Method	Relative L2: mean ± sample std	Relative H1: mean ± sample std
1D	PINN	1.035e-06 ± 4.82e-07	9.558e-06 ± 4.39e-06
1D	Deep Ritz	1.482e-05 ± 1.15e-05	1.685e-04 ± 1.24e-04
1D	VPINN	1.247e-05 ± 1.16e-05	1.444e-04 ± 1.28e-04
2D	PINN	3.804e-06 ± 1.47e-06	1.472e-05 ± 5.25e-06
2D	Deep Ritz	8.491e-05 ± 1.65e-05	3.610e-04 ± 7.61e-05
2D	VPINN	7.787e-05 ± 1.28e-05	3.968e-04 ± 1.02e-04

The 8 FEM runs demonstrate grid convergence. Maximum neural boundary error is 2.220e-16; the maximum change in relative L2 under quadrature refinement is 4.872e-12. Neural training times and CPU FEM assembly/solve times use different accounting, so they do not establish equal-hardware speedups. The finite-test-space loss of norm-aware VPINN is not, by itself, a continuous-error certificate.

2.2 Chapter 3: direct operator predictions on new tests

The main task maps reaction coefficients to solutions, retaining the fixed small diffusion coefficient. Fixed-forcing tests contain 128 samples; variable-forcing tests contain 32 new coefficient groups with 4 forcing terms each. Errors below are percentages; ± denotes population standard deviation across seeds. Single-seed entries do not estimate uncertainty.

Method/variant	Seeds	Relative L2 (%)	Relative H1 (%)
baseline/fno	3	0.4620 ± 0.0273	3.2897
baseline/deepxde_fourier4	3	7.7055 ± 0.5446	11.8322
baseline/gno	3	1.5940 ± 0.0175	21.2400
baseline/galerkin	3	0.7062 ± 0.0968	3.5672
pod/rank_4	3	4.6278 ± 0.0070	9.3134
pod/rank_8	3	2.7149 ± 0.0509	5.9411
pod/rank_16	3	2.5273 ± 0.0712	5.1668
pod/rank_32	3	2.6174 ± 0.0390	5.2824
physics/supervised_128	3	0.6849 ± 0.0565	5.7625
physics/supervised_32	3	1.7867 ± 0.1352	8.8836
physics/energy_0	3	0.4519 ± 0.0169	1.6847
physics/hybrid_32	3	0.4727 ± 0.0222	1.8211
physics/hybrid_128	3	0.4372 ± 0.0264	1.7720
varmion/varmion	3	7.3381 ± 0.0854	12.9603
varmion/unstructured	3	9.6642 ± 0.2176	11.4802
varmion_v2/dense_data	3	7.2831 ± 0.0575	12.9895
varmion_v2/spd_joint_aug128	3	3.2762 ± 0.0858	4.9198
varmion_v2/dense_joint	1	4.1956 ± 0.0000	6.6877

Continued on next page

Continued

Method/variant	Seeds	Relative L2 (%)	Relative H1 (%)
varmion_v2/spd_data	1	6.4915 ± 0.0000	11.3480
varmion_v2/spd_joint	1	5.8365 ± 0.0000	7.7016

FNO and Galerkin have relatively small L2 errors here, but DeepONet is about 7.71%, GNO H1 about 21.24%, and original VarMiON about 7.34%. Passing an audit does not erase these differences. SPD joint training with augmentation yields 3.28% for VarMiON v2, with different data, output spaces, and budgets. POD projection and exact reduced Galerkin are diagnostics, not deployable predictors.

2.3 HINTS: discrete accuracy and timing scope

Mode	Converged	Mean steps	Total ms/case	Jacobi speedup	Max. discrete L2
jacobi	64/64	4040.4	201.45	1.00	2.745e-08
fixed	192/192	422.4	34.95	5.76	3.356e-09
safeguarded	192/192	406.0	35.00	5.76	7.102e-09

All 448 equation-solver combinations converged. Each neural mode uses 64 forcing terms x 3 seeds; Jacobi uses 64 cases. Errors are relative to a direct solve of the same 65x65 P1 system. Mean continuous-reference L2 error is still about 0.09574%; discrete linear-system error is not continuous-PDE error. Timings include assembly, conditional matrices, residual conversion, and acceptance/fallback, using the median of 3 repetitions per case and excluding model loading. The approximately 5.76-fold speedup applies only to same-machine Jacobi comparisons.

2.4 Chapter 4: neural stages, head correction, and extensions

The baseline uses a fixed 17-mode linear problem family. The table selects pure-neural results for the fixed instance and the same first 8 new held-out cases. Values are relative control L2 percentages; ± is the three-seed sample standard deviation. All other stages are retained in the full evidence.

Method (neural stage)	Fixed-instance error (%)	Same 8 held-out control errors (%)
direct	4.89 ± 3.33	Fixed-instance method
kkt	0.3953 ± 0.293	Fixed-instance method
alm	0.5894 ± 0.298	Fixed-instance method
fno_state	2.174 ± 0.904	0.6436 ± 0.0641
fno_kkt	6.377 ± 4.59	1.121 ± 0.116
deeponet_state	5.71 ± 0.5	2.773 ± 0.135
deeponet_kkt	5.844 ± 1.67	5.686 ± 0.444
fedonet_state	4.157 ± 0.123	1.798 ± 0.0872
fedonet_kkt	2.269 ± 0.398	1.968 ± 0.148
mimo	2.489 ± 0.553	2.875 ± 0.468
rb_state	7.538 ± 4.55	6.377 ± 0.546
rb_dino	2.531 ± 1.13	1.675 ± 0.211

Output-head correction can approach roundoff-level control error within the declared finite modal space. This is a post-training linear-head solve, not the same accuracy attained by the pure neural model. Fixed-feature bilevel, Deep Uzawa, and FEM-NN baselines likewise are not general nonlinear networks. FEM gradients include discretization error: even the exact spectral solution has an FEM256 diagnostic of about 0.003808.

Extensions: all hidden layers are trained. Deep Uzawa uses a trainable state network and analytic control block; FEM-NN uses a trainable source network. Metrics below follow independent true-PDE solves.

Workflow	Control L2 (%)	Relative true-PDE gradient	Source L2 (%)
fem_clean	1.799 $\pm$ 0.041	0.0551	8.59
fem_noisy	3.168 $\pm$ 0.243	0.0369	17.32
fem_regularized	3.117 $\pm$ 0.220	0.0406	13.42
uzawa	1.991 $\pm$ 0.067	0.0233	-

With 1% sparse-observation noise, source error is 17.32%, falling to 13.42% with regularization. Observation fit alone does not establish correct identification. The 512 new locations provide spatial diagnostics for the same physical problem, not generalization to a new problem family. Finite inner iterations do not justify exact-convex-inner-solve convergence guarantees.

2.5 Chapter 5: constraint feasibility and optimality residuals

The 7 workflows include fixed-instance and problem-family methods. Independent Gauss control L2 errors for OSNN and ADMM-PINN are 0.0049146% and 0.00093698%, respectively. These are not ranked against the new problem-family tests below.

Method, 128 new cases	Control L2 (%)	Relative true-objective difference (%)	True projected-residual L2	Active-set error (%)
Projected KKT-FNO	4.6641 $\pm$ 0.673	0.089598 $\pm$ 0.0172	0.038412 $\pm$ 0.00602	4.0607 $\pm$ 0.732
PDOL	1.6085 $\pm$ 0.279	0.011903 $\pm$ 0.00551	0.013344 $\pm$ 0.00331	1.2885 $\pm$ 0.237
iUzawa-Net-F	1.8071 $\pm$ 0.134	0.015111 $\pm$ 0.00288	0.013703 $\pm$ 0.00112	2.1286 $\pm$ 0.226
iUzawa-Net-S	3.7659 $\pm$ 0.502	0.063337 $\pm$ 0.019	0.029252 $\pm$ 0.00451	4.106 $\pm$ 0.825

Family metrics use the same 128 cases and independent true-PDE solves; active-set tolerance is 1e-3. Surrogate fixed-point and true projected residuals are reported separately. Nonzero residuals show that completing the budget does not imply strict KKT optimality. Exact projection and positive-semidefinite preconditioners in iUzawa also do not automatically establish learned true adjoints or a whole-network convergence theorem.

CNP is a separate state-upper-bound problem: the order-96 quadrature objective is 0.073962 $\pm$ 1.76e-7, versus 0.07395772266 for the FD128 state-constrained QP reference. Control L2 error against that reference is 1.1473% $\pm$ 0.0928%. Upper-bound violations are 0 for both the neural state and the state recomputed by the true PDE. Objective differences across discrete spaces retain their signs; feasibility is not continuous optimality.

Fine-grid controls for operators use fixed-grid interpolation/edge extension. This is an independent numerical diagnostic, not network resolution transfer. Historical entry name mimo identifies Projected KKT-FNO here, not a replication of the original multiplicative MIMOONet.

3 Independent execution, selection, and protection rules

The wheel initializes an arbitrary new workspace without importing the original project or upstream repositories. Import paths were checked from /tmp. Python, PyTorch, NumPy, SciPy, and Matplotlib remain required; full Chapter 3 budgets use CUDA. The tested environment was Python 3.12.9, PyTorch 2.5.1+cu121, NumPy 2.0.1, SciPy 1.15.1, Matplotlib 3.10.1, and A100 40GB.

The independent virtual environment shared scientific libraries already installed on the server. A clean-operating-system or fully offline dependency installation was not verified. DeepXDE LGPL, Galerkin MIT, and ocpde_dl provenance are retained. Standalone does not mean wholly original or uniformly licensed.

During formal training, the 317 numerical source files and source manifest were unchanged; budgets were not reduced and old results were not substituted. Original data, results, the frozen Chapter 2 directory, and readable directories totaled 9,550 files and 7,868,164,617 bytes and matched the pretraining baseline individually. Source and artifact manifests are distinct and are not substitutes for license provenance. English presentation changes are documented separately in v1.0.1; the original manifests continue to identify v1.0.0.

Recovery protocol: Chapters 2-3 provide checkpoint recovery. Chapters 4-5 skip completed tasks only after their respective audits. Restarting a training run is not exact resumption. Do not unconditionally restart the pipeline in a completed formal workspace or remove a lock still held by a running process.

4 Book-wide manuscript-to-code checklist

The reference manuscript is a 144-page PDF. Page references use printed page numbers; PDF page order is printed page number plus 6. This mapping inherits the previously reviewed checklist: 68 algorithm/mechanism entries, not 68 disjoint leaf sections. Coverage limitations are retained; numbered sources are listed in the appendix.

Table labels: Expl. = explanatory; Rep. = representative; None = not covered. Core coverage means the main construction is implemented. Representative implementation is limited to the declared special case. Partial coverage and not covered do not indicate complete experiments. Explanatory coverage does not require separate training tasks. All implemented workflows included in the agreed formal scope of Chapters 2-5 completed this delivery.

Manuscript location	Coverage	Implementation / source index	Limitations
1.1: PDE approximation and constrained optimization, p. 1	Expl.	Book-wide index linking Chapters 2-5	Representative examples for four types of computational task
1.2: Classical baselines and motivation for neural methods, p. 2	Expl.	Chapter 2 FEM, Chapter 4 spectral/FEM, Chapter 5 QP references	Classical comparisons are available; a single example cannot establish a universal speedup
1.3: Neural-method setup, p. 4	Expl.	Consistent notation and chapter reading guides [1]	Function representations, operator surrogates, optimization, and hybrid roles are distinguished
1.4: Scope and structure, p. 5	Expl.	This checklist and the current project entry point	Four algorithmic parts and deferred branches are identified
2.1: Introduction, p. 7	Expl.	Chapter documentation	Shared examples and reading route
2.2.1: Common numerical template, p. 9	Rep.	mesh.py, fem.py [2] [3]	P1 FEM, quadrature, independent errors, and 8 grid runs; not every discretization method
2.2.2: Trainable representations, p. 12	Rep.	models.py [4]	Tanh networks, coordinate scaling, and boundary embedding; not an exhaustive study of representations
2.3.1: PINN, p. 14	Core	objectives.py [5]	6 formal runs; integrated squared strong residual; hard boundary conditions make the boundary penalty zero
2.3.2: Modifications and training issues, p. 15	Partial	models.py, training.py [4] [6]	Hard boundaries and Adam/L-BFGS are included; no full comparisons of adaptive losses or residual-adaptive sampling
2.3.3: Error analysis, p. 18	Expl.	metrics.py [7]	Independent quadrature, L2/H1 seminorms, and physical residuals; not numerical certificates for every theoretical error component
2.4.1: Deep Ritz, p. 24	Core	objectives.py [5]	6 runs; direct energy minimization without squaring the energy again; inhomogeneous 1D boundaries use an affine feasible space

Continued on next page

Continued

Manuscript location	Coverage	Implementation / source index	Limitations
2.4.2: Weak form/Petrov-Galerkin, p. 26	Rep.	As above; mesh.py supplies test bases [2]	6 norm-aware VPINN runs using a Gram-matrix weak-residual dual norm; not WAN
2.4.3: Comparison with PINN, p. 29	Core	Formal report	Shared representations, quadrature, and nominal budgets within each dimension; actual calls and costs reported separately, without equal-hardware speedup claims
2.5.1: Domain decomposition, multiscale and mixed forms, p. 30	None	Only global coordinate representations are currently used; see coverage limitations	No complete XPINN, specialized multiscale, or mixed first-order-system experiments; optional extensions
2.5.2: Architecture, regularization, and training dynamics, p. 33	Partial	training.py [6]	Fixed architectures, optimization histories, seeds, and recovery checks are available; no systematic architecture/regularization study
2.5.3: Data augmentation, p. 34	None	Current training is driven by forcing and boundaries	Analytic solutions are used only for validation; no formal sparse/noisy observation-assimilation experiments
2.6: Implementation and verification, p. 35	Core	report.py, revision review [8]	Independent quadrature, boundaries, energy consistency, FEM, and model reload checks
2.7: Strengths and limitations, p. 37	Expl.	Interpretation in the formal report	Small loss in a finite test space is not a continuous-error certificate; H1 denotes the gradient seminorm
3.1, 3.2.1: Solution operators and repeated queries	Rep.	reaction.py, reaction_data.py, ReactionOperator [9] [10] [11]	Documentation and experiments cover reaction-coefficient-to-solution maps with fixed diffusion and variable-forcing tasks; not every target operator
3.2.2: Representation, approximation, and generalization	Rep.	Data/training protocols and representation-error diagnostics [12] [13]	Train/val/test separation, normalization, and POD errors are covered; theoretical theorems are not labeled as completed training, and no universal generalization guarantee is claimed
3.3.1: DeepONet, branch-trunk contraction in Eq. (3.10)	Core	DeepXDEOperator and upstream core components [14] [15]	Delivered, 3 runs; formal ID ‘deepxde_fourier4‘; trunk coordinates include order-4 Fourier features, explicitly labeled
3.3.2: Fourier neural operator	Core	SpectralConv2d / FNO2d [16]	Delivered, 3 runs; lifting, truncated spectral convolution, local branch, and projection; same-grid error does not establish continuous-operator consistency
3.3.2: Graph neural operator, Eq. (3.16)	Core	IntegralKernel2d / GNO2d [17]	Delivered, 3 runs; 17x17 graph nodes, weighted message integration, and P1 output reconstruction; gradient error is limited by the output space
3.3.3: Transformer operators; Galerkin linear attention in Eqs. (3.21)-(3.24)	Core	GalerkinOperator and attention components [14] [18]	Delivered, 3 runs; Galerkin attention plus Fourier decoder. The manuscript also discusses generic softmax and query interactions; not every variant has a separate formal experiment
3.4.1: Physical information and mixed data/physics objectives	Rep.	TorchP1 / objective, training driver [19] [20]	Delivered, 15 runs; P1 energy-based physical FNO with data-only, energy without training labels, and hybrid training. Energy variants still use validation labels for selection

Continued on next page

Continued

Manuscript location	Coverage	Implementation / source index	Limitations
3.4.2: Reduced bases/PCA, Eq. (3.28), and fixed decoder	Core	fit_pod / CoefficientNet / reconstruct [21]	Delivered, 12 runs; training-only POD in the mass inner product at ranks 4/8/16/32. No separate residual-greedy basis-selection workflow
3.4.3: Variational structure, Eqs. (3.30)-(3.35)	Rep.	VariationalMap / UnstructuredMap [22]	Delivered, 6 runs; fixed 64-dimensional POD basis, linear load encoding, nonlinear reaction-coefficient branch, and unstructured comparison
3.4.3: Project extensions of the variational structure	Rep.	MatrixMap / operator_loss, trainer_v2.py [23] [24]	Delivered, 9 runs; dense/SPD maps and operator-consistency supervision. SPD uses additional physical information; this is not a separate manuscript-required architecture family, and not every variant has 3 seeds
3.5.1: Learning residual-to-error corrections, Eqs. (3.37)-(3.41)	Core	residual_data.py, residual-model training [25] [26]	Delivered, 3 residual models trained on solver residuals. Other missing components of this section are documented in the coverage limitations
3.5.2: Fixed HINTS and residual safeguards, Eq. (3.43), Algorithm 2	Core	hybrid.solve, test_hints [27] [13]	Delivered: Jacobi/fixed hybrid/safeguarded hybrid, totaling 448 equation-solver combinations. The same 3 residual models are reused without double-counting training
3.6: Goal-oriented implementation and verification	Rep.	heldout.py, evaluation.py, audit.py, validation.py [28] [13] [29] [30]	Independent tests, L2/H1 seminorms, physical diagnostics, model reloads, and identity audits are delivered for the current problem; not all geometries, time-dependent tasks, or out-of-distribution cases
3.7: Strengths and limitations	Expl.	Reading guide and original review report [31]	Representation error, physical-information budgets, cost, and generalization limits are explained; no additional training task
4.1: Introduction, p. 73	Expl.	Unified delivery [32]	Single-instance, problem-family, and hybrid roles are distinguished
4.2: Representative smooth problem, p. 74	Rep.	problem.py [33]	Linear-quadratic Poisson control, low-frequency state, and order-8 control; semilinear/parabolic extensions deferred
4.3: Classical numerical methods, p. 77	Core	fem.py, unified protocol [34] [35]	Classical spectral optimization, P1 FEM, and discrete adjoints; grid study at 32/64/128/256; deterministic references are not assigned artificial seeds
4.4.1: direct-PINN, p. 80	Rep.	instance.py [36]	3 runs; separate pure-neural and head-correction stages; hard boundaries, frequency enrichment, and Gauss quadrature
4.4.1: KKT-PINN, p. 80	Core	training.py [37]	3 runs; state, adjoint, and control optimality-condition training with independent true-PDE checks
4.4.1: Augmented Lagrangian, p. 80	Rep.	instance.py [36]	3 runs; outer multipliers and neural inner solves; head correction reported separately; the multiplier corresponds to the negative physical adjoint
4.4.1: Bilevel neural optimization, p. 80	Rep.	bilevel.py [38]	3 runs; actual lower-level training and implicit hypergradients, but still a linear-head baseline with 17 fixed sine features

Continued on next page

Continued

Manuscript location	Coverage	Implementation / source index	Limitations
4.4.2: FNO, DeepONet, FEDONet, p. 83	Core	models.py, training.py [39] [40]	Three architectures, each with state/KKT roles x 3 seeds = 18 runs; pure-neural and training-only head-correction stages recorded separately
4.4.2: Multi-input operator MIMOONet, p. 83	Rep.	operator_models.py, operators.py [41] [42]	3 runs; four multiplicatively fused branches and a shared trunk; modal scaling and 17-mode output projection
4.4.2: Reduced bases and derivative information, RB/DINO, p. 83	Rep.	operators.py as above [42]	3 runs each for RB-state and RB-DINO; DINO uses genuine derivative supervision within a fixed finite-dimensional space
4.4.3: Comparing neural strategies, p. 88	Rep.	Formal baseline report	Shared problem family and explicit model stages; state-surrogate control and direct control also compared on the same 8 cases
4.5.1: Deep Uzawa baseline, p. 89	Rep.	deep_uzawa.py [43]	3 runs; finite-feature baseline with inner stationarity and multiplier-update records
4.5.1: Deeper Deep Uzawa representations, p. 89	Rep.	Extended training and documentation [44]	3 new runs; coordinate networks with all hidden layers trainable, but analytic quadratic control updates; not a replication of exact-inner-solve theory
4.5.2: ISMO, p. 93	Core	ismo.py, hybrids.py [45] [46]	3 runs; nonlinear surrogate retrained after active sampling with the true PDE; cheap spectral PDE solves do not support general end-to-end speedup claims
4.5.3: FEM-NN baseline, p. 96	Rep.	fem_nn.py [47]	3 runs; fixed-feature source identification with noiseless full-field observations; FEM solves the state
4.5.3: Extended FEM-NN observations, p. 96	Rep.	Extended training and independent evaluation [44] [48]	9 new runs: sparse noiseless, 1% noise, and the same noise plus regularization, each with 3 seeds; trainable source network and sparse transpose backpropagation
4.6: Implementation and verification, p. 100	Core	Baseline assessment and extension report [49]	Source/model freezing, true-PDE solves, objectives/gradients, grid/boundary checks, and reload audits
4.7: Discussion and open questions, p. 102	Expl.	Chapter 4 guide and two formal reports [50]	Representation-space, head-correction, finite-inner-iteration, and cost limitations are retained
5.1: Introduction, p. 105	Expl.	Unified-delivery documentation [51]	Control constraints, state constraints, interfaces, and lower-level variational inequalities are distinguished
5.2: Nonsmooth objectives, p. 107	Partial	Problem definitions and Sections 5.2.1/5.2.2 workflows [52]	Experiments cover control box constraints and state upper bounds; general nonsmooth and sparse control are not fully covered
5.2.1: OSNN, p. 107	Core	models.py, training [53] [54]	3 runs; state/scaled-adjoint networks with exact box projection to recover control
5.2.1: Control-splitting ADMM-PINN, p. 107	Core	SplitState and training as above [54]	3 runs; neural inner solves, split variables, and multiplier histories retained; the PDE uses inner control u, while feasible output z is reported separately
5.2.1: Projected KKT-FNO, p. 107	Rep.	operators.py [55]	3 runs; historical ID mimo; not the multiplicative-branch MIMOONet from Chapter 4

Continued on next page

Continued

Manuscript location	Coverage	Implementation / source index	Limitations
5.2.1: PDOL, p. 107	Core	operators.py, reference.py [55] [56]	3 runs; state/adjoint FNO surrogates in primal-dual iterations; surrogate fixed-point and true-PDE residuals separated
5.2.1: iUzawa-Net-F/S, p. 107	Core	operator_blocks.py [57]	Two weight-unrolling variants x 3 seeds = 6 runs; exact projection and structured positive-semidefinite preconditioners; no whole-network convergence guarantee
5.2.1: L1 sparse control, p. 107	Partial	Proximal tools and tests in the original modules; see coverage review	Basic tools only; no complete sparse-control training, reference, or independent evaluation
5.2.2: CNP state constraints, p. 112	Rep.	models.py, independent evaluation [53] [58]	3 runs; smooth feasible states, true Laplacian control recovery, and an independent state-constrained QP; not obstacle control
5.2.2: State-splitting ADMM, p. 112, Eqs. (5.19)-(5.21)	None	No complete entry point yet; see coverage limitations	Deferred; neither control-splitting ADMM nor CNP replaces this branch
5.3.1: Interface models, p. 116	None	No formal implementation yet	Deferred; piecewise PDEs, one-sided fields, value/flux jumps, and independent classical references are not delivered
5.3.2: Neural interface representations, p. 118	None	No formal implementation yet	Deferred; DCSNN soft constraints, lifting/hard embedding, and interface diagnostics are not delivered
5.4.1: Variational inequalities and obstacle control, p. 122	None	No formal implementation yet	Deferred; lower-level obstacle solves, complementarity, and independent control references are not delivered
5.4.2: Neural bilevel formulations, p. 123	None	No formal implementation yet	Deferred; Chapter 4 smooth finite-feature bilevel methods and CNP cannot replace this branch
5.4.3: Moreau envelopes and S2-FOBA, p. 124	None	No formal implementation yet	Deferred; envelope-difference terms, the two-stage algorithm, and fixed-control state refinement are not delivered
5.5: Implementation and verification, p. 126	Partial	protocol.py, assessment.py [59] [58]	Physical, feasibility, active-set, and reload diagnostics cover the seven current Section 5.2 workflows, not deferred problems
5.6: Discussion and open questions, p. 127	Expl.	Formal report and reading guide [60]	Boundaries between current experiments and more general nonsmooth problems are explicit
6: Conclusions and outlook, pp. 129-132	Expl.	Summary of Chapters 2-5 methods, validation, and limitations	No additional neural training; unimplemented directions are not counted as complete

Still deferred: Chapter 5 state-splitting ADMM, interface control, obstacle control/S2-FOBA, and semilinear/parabolic extensions. Neural initialization + CG has left the main scope; Darcy replication is paused. Control-splitting ADMM does not replace state-splitting ADMM; CNP does not replace obstacle control.

5 Review corrections and evidence index

5.1 Historical evidence and two provenance clarifications

The original Chapter 4 summary.md retains inherited template text claiming 9 reused seed0 runs. In this delivery, adopted_jobs.json is empty; all 27 original-implementation tasks and 21 additional tasks have training records, and the pipeline passed no reuse option. All 48 runs were trained from scratch. The template sentence does not apply. The original summary and FINAL_AUDIT retain their historical identities in v1.0.0; summary.reviewed.md and REVIEW_NOTES_ZH.md supply the correction. This report uses their numbers without repeating the incorrect provenance claim. English translations have separately recorded new identities.

INSTALLATION.json describes the early wheel and 314 sources from the first installation. The pretraining update `PRETRAIN_SOURCE_UPDATE.json`, DEPENDENCIES.json, and the formal 317-file manifest identify the version actually trained. The early installation hash is not the final wheel hash. The original final identities are listed below; v1.0.1 English-edition identities are recorded in its Release checksums and localization manifest.

5.2 Identities and evidence paths

Paths below are relative to the complete independent workspace R, originally `/home/yhr/Desktop/NeuralPDE-Opt-independent-v1`. The local `standalone/delivery/workspace` initially mirrored sources and evidence only; complete checkpoints remained on the server. Release attachments now also provide the 180 selected models and final-test arrays, excluding original training/validation samples and recovery checkpoints.

Identity	SHA256
Manuscript book.pdf	`85191f64ad9f904afa4aed86ce2dcd4bdfc272b2aec21d2c649261c7ef5670e2`
Original final wheel v1.0.0	`f7c59bec496898be46b223f4d401b66091c9d65b9b611be0b7b834bdcf9534a9`
PAYLOAD_MANIFEST.json	`e07d6acdeb2a9365952196f996ef8c8c039917f39a219097a8aa847dd77afef6`
FINAL_AUDIT.json	`633c293ff1e5ddfa054555eddab012d75d8d5289fe9a50c465b4430cb5e87598`

Content	Path relative to R
Pipeline and exit status	`verification/PIPELINE.json`; `verification/pipeline_PROCESS.json`
Regression/development checks	`verification/CHECKS.json`; `verification/SMOKE.json`
Pretraining update and dependencies	`verification/PRETRAIN_SOURCE_UPDATE.json`; `verification/DEPENDENCIES.json`
Book-wide audit and re-verification	`reports/FINAL_AUDIT.json`; `verification/FINAL_DELIVERY_RECHECK.json`
Original frozen-asset protection	`verification/ORIGINAL_ASSETS.json`; `verification/ORIGINAL_ASSETS_FINAL_CHECK.json`
Chapter 2	`results/ch02/full/summary.md`; `results/ch02/full/DELIVERY_AUDIT.json`
Chapter 3 training and testing	`results/ch03/final_v1_support/training_audit.json`; `results/ch03/final_test_v1/verification.json`
Chapter 4 baseline	`results/ch04/unified_delivery_v1/DELIVERY_AUDIT.json`; `results/ch04/unified_delivery_v1/summary.reviewed.md`
Chapter 4 extensions	`results/ch04/nonlinear_depth_v1/DELIVERY_AUDIT.json`
Chapter 5	`results/ch05/unified_delivery_v1/DELIVERY_AUDIT.json`
Review notes	`reports/REVIEW_NOTES_ZH.md`

5.3 Reproduction in a new workspace

The following commands apply to a separate new workspace, not the completed delivery directory. The complete run uses formal budgets, and initialization refuses to overwrite an existing directory.

```
python -m pip install neuralpde_book-1.0.1-py3-none-any.whl
neuralpde-book init /absolute/path/new-workspace
neuralpde-book check --workspace /absolute/path/new-workspace
neuralpde-book prepare --workspace /absolute/path/new-workspace
neuralpde-book smoke --workspace /absolute/path/new-workspace
neuralpde-book pipeline --workspace /absolute/path/new-workspace
neuralpde-book status --workspace /absolute/path/new-workspace
```

6 Source index

Paths are relative to R; numbers correspond to the book-wide mapping table. The index identifies the 317-file source inventory used in training. English presentation changes do not alter numerical algorithms; original training identities are retained separately.

No.	Relative source path
[1]	README.md
[2]	src/neuralpde_ch02/mesh.py
[3]	src/neuralpde_ch02/fem.py
[4]	src/neuralpde_ch02/models.py
[5]	src/neuralpde_ch02/objectives.py
[6]	src/neuralpde_ch02/training.py
[7]	src/neuralpde_ch02/metrics.py
[8]	src/neuralpde_ch02/report.py
[9]	src/neuralpde_opt/chapter3/reaction.py
[10]	src/neuralpde_opt/chapter3/reaction_data.py
[11]	src/neuralpde_opt/chapter3/reaction_operators.py
[12]	src/neuralpde_opt/chapter3/reaction_training.py
[13]	chapters/ch03/evaluation.py
[14]	src/neuralpde_opt/chapter3/upstream_operators.py
[15]	src/neuralpde_opt/chapter3/upstream_vendor/deepxde_core.py
[16]	src/neuralpde_opt/chapter3/fno.py
[17]	src/neuralpde_opt/chapter3/gno.py
[18]	src/neuralpde_opt/chapter3/upstream_vendor/galerkin_layers.py
[19]	experiments/ch03_physics_v1/physics.py
[20]	experiments/ch03_physics_v1/run.py
[21]	experiments/ch03_pod_v1/pod.py
[22]	experiments/ch03_varmion_v1/model.py
[23]	experiments/ch03_varmion_v2/model_v2.py
[24]	experiments/ch03_varmion_v2/trainer_v2.py
[25]	experiments/ch03_hints_v1/residual_data.py
[26]	experiments/ch03_hints_v1/run.py
[27]	chapters/ch03/hybrid.py
[28]	chapters/ch03/heldout.py
[29]	chapters/ch03/audit.py
[30]	chapters/ch03/validation.py

Continued on next page

Continued

No.	Relative source path
[31]	docs/CH03_READING_ZH.md
[32]	chapters/ch04_delivery/README.md
[33]	chapters/ch04/problem.py
[34]	chapters/ch04/fem.py
[35]	chapters/ch04_delivery/protocol.py
[36]	chapters/ch04_adaptation/instance.py
[37]	chapters/ch04/training.py
[38]	chapters/ch04_completion/bilevel.py
[39]	chapters/ch04_operator_adaptation/models.py
[40]	chapters/ch04_operator_adaptation/training.py
[41]	chapters/ch04_adaptation/operator_models.py
[42]	chapters/ch04_delivery/operators.py
[43]	chapters/ch04_completion/deep_uzawa.py
[44]	chapters/ch04_depth/training.py
[45]	chapters/ch04_completion/ismo.py
[46]	chapters/ch04_delivery/hybrids.py
[47]	chapters/ch04_completion/fem_nn.py
[48]	chapters/ch04_depth/assessment.py
[49]	chapters/ch04_delivery/assessment.py
[50]	docs/CH04_READING_ZH.md
[51]	chapters/ch05_delivery/README.md
[52]	chapters/ch05/problem.py
[53]	chapters/ch05/models.py
[54]	chapters/ch05_delivery/training.py
[55]	chapters/ch05/operators.py
[56]	chapters/ch05/reference.py
[57]	chapters/ch05/operator_blocks.py
[58]	chapters/ch05_delivery/assessment.py
[59]	chapters/ch05_delivery/protocol.py
[60]	docs/CH05_READING_ZH.md